

D-FMA: Dynamic Feature-Mixing Attack

Adapting Semantic Morphing for Enhanced Transferability in Face Verification

Puneet Kumar (Indian Institute of Information Technology, Senapati,
Manipur)

June 2026

Why Dynamic Feature-Mixing (D-FMA)?

- **Literature Inspired:** Directly adapts concepts from recent semantic and Attention-Aware adversarial literature (e.g., *TransMix*, 2024).
- **The Flaw in Baselines:** Vanilla attacks (PGD, MI-FGSM) rely on global L_∞ pixel noise. This overfits to the surrogate model's spatial layout and is easily mitigated by robust verification systems.
- **The Solution:** D-FMA disrupts the fundamental semantic geometry of the face (eyes, nose) before optimization, destroying the victim model's primary spatial anchors.

How D-FMA Achieves Semantic Disruption:

- 1 **Geometric Feature Targeting:** Maps the exact semantic coordinates of the highly-discriminative periorcular (eye) and nasal regions.
- 2 **Mathematical Feature Swapping:** Physically blends the Target identity's features into the Source identity's skull structure.
- 3 **Gaussian Alpha-Blending:** Applies an extreme spatial blur (`cv2.GaussianBlur`) to the masking tensor. This ensures injected features blend seamlessly into the surrounding skin, neutralizing high-frequency edge artifacts.
- 4 **Optimization:** Wraps the morphed tensor in an MI-FGSM loop to accumulate highly transferable adversarial gradients directly onto the mixed features.

Adapting to the Face Verification Codebase

Code Integration Steps:

- **Step 1 - Dependency Injection:** Integrated OpenCV (cv2) and NumPy array manipulation into the eager TensorFlow core engine.
- **Step 2 - Custom Architecture:** Engineered the `dynamic_morph_mi_fgsm()` function to apply semantic blending *before* the `tf.GradientTape` loss calculation.
- **Step 3 - Dispatcher Update:** Appended 'DYNAMIC_MORPH' to `ALL_ATTACKS` in `transfer_attack_core.py`.
- **Step 4 - Evaluation Pipeline:** Modified `build_subset_baselines.py` to ingest and evaluate newly registered custom attack arrays without silently dropping unrecognized paths.

Final Results: Dodging & Impersonation

Evaluation on 4x Surrogate and 5x Victim Architectures:

Attack Method	Dodging Success	Impersonation Success
DYNAMIC_MORPH (Proposed)	45.83%	31.94%
SI-NI-FGSM	38.75%	24.17%
MI-FGSM	33.75%	18.33%
MI-ADMIX-DI-TI	30.42%	17.50%
TI-FGSM	26.67%	13.33%
PGD	22.08%	10.83%

Conclusion: D-FMA completely shattered the existing baselines, setting a new State-of-the-Art on this subset (+7.08% in Dodging and +7.77% in Impersonation over the top baseline, SI-NI-FGSM). **Github link:**

"<https://github.com/puneetkumar-2005/feature-mixing-transfer-attack.git>"

What I Learned

Technical Takeaways:

- Integrating C++ backed OpenCV logic with eager TensorFlow execution requires extremely careful type casting and shape manipulation to avoid crash loops.
- Dynamic landmark detection is incredibly fragile on highly-cropped localized datasets; robust fallback mechanisms (or static pre-aligned masks) are vital for batch processing.

Process Takeaways:

- Shifting attack strategies from pure pixel-level noise to structural/semantic manipulation yields massive, double-digit gains in transferability.
- Debugging evaluation infrastructure is critical: custom attacks will silently fail if hardcoded benchmarking scripts aren't updated to recognize them.